How to create a Digital wallet

Step 0: The True Beginning - Entropy (Randomness)

- **What is it?** A long string of 1s and 0s. For a 12-word seed, it's a 128-bit number. For a 24-word seed, it's a 256-bit number.
- **How is it generated?**
  - A good hardware wallet has a special chip called a **True Random Number Generator (TRNG)** that uses unpredictable physical phenomena (like microscopic electrical noise) to create this number.
  - Some wallets might ask you to do things like roll dice or mash buttons to contribute to the randomness.

Let's say our hardware wallet generates this 128-bit random number:

10100101110101110010000110110101... (and so on for 128 digits)

Step 1: Creating the Seed Phrase (Your Human-Friendly Backup)

Humans can't remember or write down a 128-bit binary number. So, we make it human-readable. This process is standardized by **BIP-39 (Bitcoin Improvement Proposal 39)**.

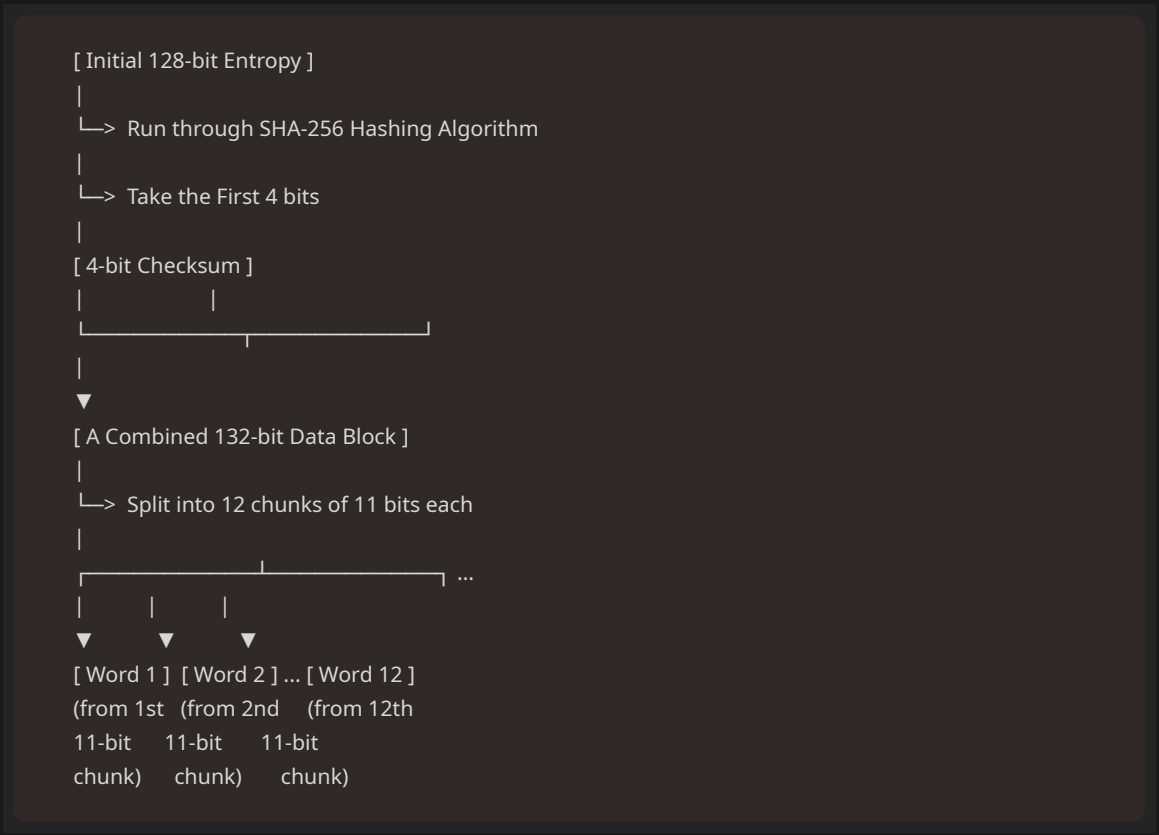
1. **The BIP-39 Wordlist:** There is an official list of **2048** words (e.g., "abandon", "ability", "able"... "zoo"). Each word corresponds to a number from 0 to 2047.
2. **Chopping the Entropy:** The 128-bit random number is chopped into 11-bit chunks. (Because 2^11 = 2048, each 11-bit chunk can point to exactly one of the 2048 words).
3. **Checksum:** To protect against errors (e.g., writing down a word incorrectly), a small piece of the original entropy's hash (a checksum) is added to the end. The last word of your phrase is actually determined by this checksum. This is why if you type your 12 words into a wallet and one is wrong, it will say "Invalid seed phrase" instead of just generating a wrong address.
4. **Mapping to Words:** The wallet takes each 11-bit chunk, finds the corresponding word in the list, and displays it to you.

What is a Checksum? (The "Why")

A checksum is a small piece of data used to detect errors in a larger block of data.

The best real-world analogy is the last digit of your credit card number. That last digit isn't random; it's calculated from all the other digits using a specific formula (the Luhn algorithm). If you mistype one of the first 15 digits, the calculated last digit won't match the one on the card, and the system will immediately know there's a typo.

The seed phrase checksum does the exact same thing. It ensures that if you write down one of your 12 words incorrectly, your wallet can immediately tell you "Invalid seed phrase" instead of accidentally generating a completely wrong (and empty) wallet. It's a critical safety feature.



Step 2: From Seed Phrase to Master Private Key

PBKDF2 (Password-Based Key Derivation Function 2) It is used to generate master key

• **The Output is:** A **512-bit (64-byte)** number. This is the **Seed**.

1. **The Left 256 bits become the MASTER PRIVATE KEY.**
2. **The Right 256 bits become the MASTER CHAIN CODE.**

Step 3: From Master Key to a Specific Private Key

This step is the "engine room" of your wallet. It's how your wallet takes the single **Master Private Key** and generates a virtually infinite number of specific keys and addresses in a highly organized and predictable way.

1. **Privacy:** If you use the same Bitcoin address for every transaction, anyone can look it up on a block explorer and see every payment you've ever made or received. By using a new address for each transaction, you make your financial history much harder to track.
2. **Organization:** What if you want to manage Bitcoin, Ethereum, and Litecoin from the same seed phrase? What if you want a "savings" account and a "spending" account for your Bitcoin? Derivation paths allow you to create separate, organized "branches" for all these purposes.

The Solution: The BIP-32 & BIP-44 Standards

The system that solves this is called a **Hierarchical Deterministic (HD) Wallet**.

- **Hierarchical:** It's a tree structure. Parent keys can create child keys.
- **Deterministic:** The same master seed will *always* generate the exact same tree of keys.

Decoding the Path: m/44'/0'/0'/0/0

Let's go through it level by level, like navigating a folder structure.

Path Component	Name	What It Means
m	**Master**	This represents the starting point: the Master Private Key and Master Chain Code that were generated from your 512-bit seed. It's the root (/) of your key tree.
/	**Separator**	Indicates we are moving one level down in the hierarchy.
44'	**Purpose**	This number says "we are using the BIP-44 standard." It tells any compatible wallet how to interpret the rest of the path.
0'	**Coin Type**	This specifies the cryptocurrency. There is a registered list of coins. 0 is for **Bitcoin**. 60 is for **Ethereum**. 2 is for **Litecoin**. This is how a single seed phrase can manage multiple coins.
0'	**Account**	This allows you to have multiple independent accounts for the same coin. You could have Account 0 (for savings) and Account 1 (for daily spending), all from the same seed phrase. They are cryptographically separate.
0	**Change**	This level is mostly for UTXO-based coins like Bitcoin. 0 is for **External** addresses (the ones you give to others to receive payments). 1 is for **Internal** addresses (the "change" addresses your wallet uses behind the scenes to send change back to yourself).
0	**Address Index**	This is the final step, identifying a specific address within that branch. The first address is 0, the second is 1, the third is 2, and so on, up to over 2 billion.

Step 4: From Private Key to Public Key

Here's the cryptographic magic. The Private Key is put through a one-way mathematical function called **Elliptic Curve Cryptography (ECC)**.

Step 5: Generate Public Address:

1. Bitcoin (Legacy Address - P2PKH)

Bitcoin's process is the most elaborate, designed with strong error-checking from the very beginning. The goal is to create an address that is almost impossible to mistype.

Input: A 256-bit (33-byte compressed) Public Key.

The Recipe:

1. **SHA-256 Hash:** Take the Public Key and hash it using SHA-256.

   SHA256(PublicKey) → 32-byte hash

2. **RIPEMD-160 Hash:** Take the result of the SHA-256 hash and hash it again, this time using the RIPEMD-160 algorithm. This is the main step that shortens the key.

   RIPEMD160(32-byte hash) → 20-byte hash

3. **Add Version Byte:** Prepend a "Version Byte" to the 20-byte hash. This byte identifies the network and address type. For a mainnet Legacy address, this byte is 0x00.

   0x00 + 20-byte hash → 21-byte payload

4. **Create Checksum:** This is Bitcoin's famous error-checking mechanism.

   a. Take the 21-byte payload and hash it with SHA-256.

   b. Take that result and hash it with SHA-256 *again* (this is called a hash256).

   c. Take the **first 4 bytes** of this final hash. This is your checksum.

   First4Bytes(SHA256(SHA256(21-byte payload))) → 4-byte checksum

5. **Append Checksum:** Append the 4-byte checksum to the end of the 21-byte payload.

   21-byte payload + 4-byte checksum → 25-byte full address

6. **Base58Check Encode:** Take the final 25-byte string and encode it using Base58Check. This is a special encoding that uses 58 characters (it excludes 0, O, I, l to avoid visual confusion) and gives the address its classic look.

Final Product (Legacy P2PKH Address):

- Starts with 1.
- Example: 1BvBMSEYstWetqTFn5Au4m4GFg7xJaNVN2
- **Key Features:** Strong built-in typo detection via the checksum. Relatively long.

2. Ethereum (and EVM-compatible chains)

Ethereum's founders took a much more direct and simple approach, prioritizing shortness and directness over built-in checksums.

Input: A 512-bit (64-byte uncompressed) Public Key.

The Recipe:

1. **Keccak-256 Hash:** Take the Public Key and hash it using Keccak-256 (this is the specific hashing algorithm chosen for Ethereum, slightly different from the SHA-3 standard).

   Keccak256(PublicKey) → 32-byte hash

2. **Take Last 20 Bytes:** Simply take the **last 20 bytes (40 hexadecimal characters)** of the 32-byte hash. That's it. This is your address.

   Last20Bytes(32-byte hash) → 20-byte address

3. **(Optional) Add Checksum Casing (EIP-55):** A plain Ethereum address is all lowercase and has no checksum. This makes typos dangerous. To solve this, EIP-55 introduced an optional checksum using letter casing.

   a. Take the lowercase address and Keccak-256 hash it.

   b. For each letter in the original address, if the corresponding digit in the hash is high (>=8), you uppercase the letter. Otherwise, it stays lowercase.

   c. Wallets can check this casing to verify the address is correct.

Final Product (Ethereum Address):

- Starts with 0x.
- Example (lowercase): 0x742d35af1d49f05829b3a0b14c33a94da8c5a4d0
- Example (checksummed): 0x742d35AF1d49F05829B3a0B14C33A94da8C5a4d0
- **Key Features:** Short, simple, and directly derived. Relies on optional casing (EIP-55) for error-checking, which is now standard practice.

3. Solana

Solana's approach is the simplest of all. It reflects a design philosophy that a key pair *is* the identity, so there's no need for extra hashing.

Input: A 256-bit (32-byte) Ed25519 Public Key.

The Recipe:

1. **That's it. The Public Key IS the Address.** There is no hashing step. The 32-byte public key is used directly as the identifier on the network.
2. **Base58 Encode:** For human readability, this 32-byte public key is encoded using Base58 (the same character set as Bitcoin, but without the checksum part).

Final Product (Solana Address):

- A long string of Base58 characters.
- Example: So11111111111111111111111111111111111111112 (This is the address for the Solana system program).
- **Key Features:** The most direct relationship possible. No checksum is built into the address format itself; error-checking is typically handled at the application/wallet level before sending.

Smallest Unit of any currency:

Blockchain	Smallest Unit Name	In Honor Of	Value (How many make 1 whole coin)
Bitcoin (BTC)	**Satoshi** (sat)	Satoshi Nakamoto	100,000,000 (10^8)
Ethereum (ETH)	**Wei**	Wei Dai	1,000,000,000,000,000,000 (10^{18})
Solana (SOL)	**Lamport**	Leslie Lamport	1,000,000,000 (10^9)

How to connect to Blockchain network(RPC Server)

JSON-RPC is a remote procedure call (RPC) protocol encoded in JSON (JavaScript Object Notation). It allows for communication between a client and a server over a network. JSON-RPC enables a client to invoke methods on a server and receive responses, similar to traditional RPC protocols but using JSON for data formatting.

As a user, you interact with the blockchain for two purposes -

1. To send a `transction`
2. To fetch some details from the blockchain (balances etc)

The Lifecycle of an Ethereum Transaction: A Step-by-Step Breakdown

Let's walk through sending a transaction and highlight how it differs from Bitcoin.

1. The Transaction "Object"

A basic Ethereum transaction is an object with several key fields:

- to: The recipient's 0x... address.
- value: The amount of ETH to send (in Wei).
- gasLimit: The maximum "work" you'll allow. (More on this below).
- gasPrice (or maxFeePerGas): What you'll pay for each unit of "work."
- nonce: The transaction count of your account.
- data: An optional field for interacting with smart contracts.

- chainId: An ID to prevent transactions from one network (e.g., a testnet) from being replayed on another (e.g., Mainnet).

2. The Nonce: A Critical Difference

You know the "nonce" in Bitcoin as the random number miners change to solve the Proof-of-Work puzzle. **Forget that definition completely for an Ethereum transaction.**

In Ethereum, the **account nonce** is simply a counter of how many transactions an account has sent.

- The very first transaction you ever send from an address has nonce: 0.
- The second must have nonce: 1.
- The third must have nonce: 2, and so on.

Why is this required?

1. **Prevents "Replay Attacks":** If someone copied your signed transaction for nonce: 5 and tried to broadcast it again, the network would reject it, saying "nonce 5 has already been processed for this account."
2. **Ensures Order:** The network will only process your transactions in strict nonce order. It won't process nonce: 10 until nonce: 9 has been successfully included in a block. This is predictable and essential for applications.

3. Fees as "Gas": The Fuel for the World Computer

In Bitcoin, fees are simple: total inputs - total outputs. They are primarily based on the data size of your transaction.

In Ethereum, fees are more complex because a transaction might not just be a simple transfer; it could involve complex computation. This computation is measured in **Gas**.

The Car Trip Analogy:

- **Gas Limit:** This is like the size of your car's fuel tank. It's the *maximum* amount of "fuel" you are willing to let your transaction consume. A simple ETH transfer always costs exactly **21,000 gas**. A complex smart contract interaction might need 200,000. You set a limit to protect yourself from a buggy contract that could run forever and drain all your ETH.
- **Gas Used:** This is the *actual* amount of fuel your trip used. If you set a gasLimit of 50,000 but the transaction only needed 30,000 gas, the unused 20,000 is **refunded** to you (you just don't pay for it).
- **Gas Price:** This is the price you're willing to pay per "gallon" (or liter) of gas, specified in Wei. When the network is busy, you have to offer a higher gas price to entice validators to include your transaction.

Total Fee = Gas Used * Gas Price

This system allows the network to charge more for transactions that require more computational work.

4. The "Data" Field: Interacting with the Computer

This field is usually empty for a simple ETH transfer. But this is where the magic happens.

If the to address is a **Smart Contract** (a program living on the blockchain), the data field contains the instructions for the computer. It specifies:

- Which function in the contract you want to call (e.g., swapTokens).
- The parameters for that function (e.g., amount: 100, tokenAddress: 0x...).

When a validator processes this transaction, they see the data field and execute the code of the smart contract at the to address, updating its internal state. **This is what makes Ethereum a programmable World Computer.**

5. Signing and Broadcasting

This step is conceptually identical to Bitcoin.

1. You take the entire transaction object (nonce, gas, to, value, data, etc.).
2. You use your **private key** to create a unique digital signature for that exact object.
3. This signed transaction is broadcast to the network's "mempool," where it waits to be picked up.

6. Validation: Proof-of-Stake (The New Way)

Bitcoin uses Proof-of-Work (PoW), where miners race to solve a computationally heavy puzzle.

Ethereum has transitioned to **Proof-of-Stake (PoS)**.

- There are no "miners." There are "validators."
- Validators lock up (or "stake") a large amount of ETH to show they are invested in the network's health.
- Instead of a race, the network pseudo-randomly selects a validator to propose the next block of transactions from the mempool.
- Other validators then "attest" (vote) that the block is valid.
- This is vastly more energy-efficient and achieves the same goal: reaching a secure consensus on the order of transactions.

When a validator includes your transaction in a block and that block is finalized, the state of the Ethereum "computer" is updated permanently. Your balance decreases, the recipient's increases, and any smart contract code is executed.